

LECTURE 5

The Relational Model

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

Agenda

- Overview of the relational model — what Codd proposed, and why it won
- The notion of a relation: attributes, tuples, degree, cardinality
- Mathematical relations vs. database relations
- The four properties every valid relation must satisfy
- Attributes and domains
- The key hierarchy: superkey, candidate key, primary key, alternate key, foreign key
- Relational database schema vs. database instance
- Standard notation for writing a relation schema

The Relational Model, in One Idea

Store data as **tables**. Let mathematical rules — not insertion order, not physical storage — define what a valid table looks like and how tables relate.

Proposed by E. F. Codd, 1970

“A Relational Model of Data for Large Shared Data Banks” replaced hierarchical/network models that forced programmers to navigate hard-coded physical paths.

Branch (branchNo, street, city, postcode)

branchNo	street	city	postcode
B003	163 Main St	Lahore	54000
B005	22 Deer Rd	Karachi	74200
B007	16 Argyll St	Islamabad	44000

Staff (staffNo, name, position, salary, branchNo)

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SL41	Julie Lee	Manager	22000	B005

Running example: a property rental agency — Branch, Staff, and the properties each branch manages — carried across Lectures 5–8.

The Notion of a Relation

A relation is a table of values, organized into rows and named columns.

Name — unique within the schema (Staff, Branch)

Attributes — named columns, each with a domain of legal values

Tuples — rows, each holding one value per attribute

Staff — degree 5, cardinality 5

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SL41	Julie Lee	Manager	22000	B005

Degree = number of attributes = 5

Cardinality = number of tuples = 5 (changes on every insert/delete)

Table, relation, or relvar?

Precisely, a relation is a value — one snapshot of tuples. What CREATE TABLE actually makes is a relation variable (relvar), whose value changes as rows are added or removed.

Mathematical Relations vs. Database Relations

“Relation” comes from set theory — a subset of a Cartesian product $D_1 \times D_2 \times \dots \times D_n$. A database relation tightens several rules that plain math relations don’t enforce.

	Mathematical relation	Database relation
Tuple order	Significant — $(1,2) \neq (2,1)$	Irrelevant — a relation is a set
Column identity	By position only	By name (branchNo, salary)
Duplicate tuples	Allowed (bag-like in some treatments)	Never allowed — a true set

Atomicity is stricter than it sounds telNo: “042-121-1121, 042-121-1122” looks harmless as a string, but violates atomicity in spirit — the DBMS can’t search or constrain “one of this client’s phone numbers” without parsing it. Fix: a separate ClientPhone(clientNo, telNo) relation, one tuple per number.

Relation Properties — the Four Rules

1

Unique name

Distinct from every other relation in the schema.

2

No duplicate tuples

A relation is a mathematical set — sets cannot repeat an element.

3

Tuple order irrelevant

Rows in any order are the same relation — ORDER BY imposes display order only.

4

Attribute order irrelevant

A value is identified by attribute name, not column position.

5

Every value is atomic

One indivisible value per cell — never a list or nested table.

Attributes and Domains

Attribute

A named column representing one property of the thing the relation describes.

Domain

The set of all legal values for an attribute — a reusable data type with business meaning attached.

Shared domains make joins meaningful

Staff.branchNo and Branch.branchNo share the BranchNumbers domain despite matching names — that shared domain is exactly what makes comparing (and later joining, Lecture 8) them meaningful.

Domain definitions used by Staff and Branch

Domain	Meaning	Example values
StaffNumbers	valid staff IDs	SL21, SG37
StaffNames	person names	John White
StaffPositions	valid job titles	Manager, Assistant
Salaries	PKR 8,500–40,000	9000, 30000
BranchNumbers	valid branch IDs	B003, B005

Relational Keys — the Hierarchy

Superkey

Any attribute set that uniquely identifies a tuple — may carry extra attributes



Candidate key

A minimal superkey — remove one attribute and uniqueness breaks



Primary key

The candidate key the designer chooses to identify tuples

`{staffNo}` is a candidate key of Staff: minimal, and unique. `{staffNo, name}` is a superkey but *not* a candidate key — it isn't minimal.

Alternate key

Every candidate key not chosen as primary — still unique, just not the schema's lead identifier.

Foreign key

An attribute set in one relation that reproduces another relation's primary key, to represent a relationship. Staff.branchNo references Branch.branchNo.

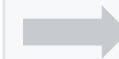
Branch

branchNo

street

city

postcode



Staff

staffNo

name

position

salary

branchNo

Testing for Candidate Keys

A quick test for candidate keys

- 1. Uniqueness** Can two distinct tuples share the same value(s) for K? If yes, K is not even a superkey.
- 2. Minimality** Drop one attribute from K — does uniqueness still hold? If yes, K was not minimal.

Applied to PropertyForRent: `{propertyNo}` is a candidate key;
`{propertyNo, street}` is a superkey but not minimal — not a candidate key.

Composite primary keys underline as a group

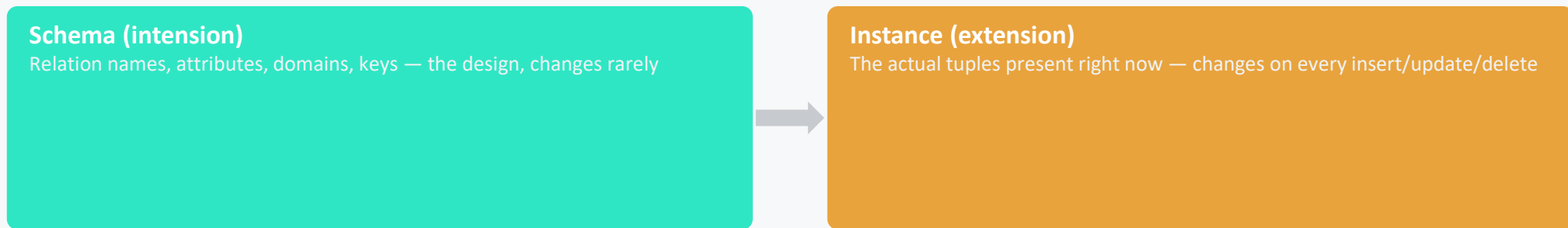
Not every primary key is one attribute.

```
Viewing (clientNo, propertyNo, viewDate,  
comment)
```

Neither clientNo alone nor propertyNo alone is unique — one client views many properties, one property is viewed by many clients — but the pair is. Convention underlines the whole pair together.

Relational Database Schema vs. Instance

A relational database schema is the full structural design: every relation schema (names, attributes, domains, keys) plus every integrity constraint (Lecture 6). The actual rows at any moment are the instance (extension).



By Lecture 8 the rental agency's schema grows to include `PropertyForRent`, `PrivateOwner`, and `Client` — giving the relational-algebra examples something richer to query.

Writing a Relation Schema

```
RelationName (attribute1, attribute2, ..., attributeN)
```

Primary key underlined; foreign keys marked in italics or called out explicitly.

```
Branch (branchNo, street, city, postcode)
```

```
Staff (staffNo, name, position,  
       salary, branchNo)
```

Calling out a foreign key explicitly:

```
Staff (staffNo, name, position, salary, branchNo)  
    Foreign Key branchNo references Branch(branchNo)
```

Memorize this notation cold — every relational-algebra expression from Lecture 7 onward operates on relations named exactly this way.

KEY TAKEAWAYS

The Relational Model

- The relational model represents data as named, independent tables linked purely through shared attribute values — no pointers, no nesting.
- A relation is a named set of tuples over fixed named attributes; degree = attribute count, cardinality = current tuple count.
- Database relations tighten mathematical relations: no duplicate tuples, no meaningful order, every value atomic.
- The key hierarchy nests: candidate key is a minimal superkey; primary key is the chosen candidate key; unchosen ones are alternate keys; a foreign key reproduces another relation's primary key.
- A schema is the full structural design; the instance is the actual rows present right now. Schemas are written `Relation(attr1, attr2, ...)` with the primary key underlined.